Guía paso a paso de la práctica

AGR — Práctica 2: Vagrant (parte 1) y red del laboratorio 3103 con QEMU-KVM (parte 2)

Pablo Elvira Calatayud

Índice

- Cómo leer esta guía
- PARTE 1 — Servidor web con Vagrant
  - P1.1. Instalar VirtualBox y Vagrant
  - P1.2. El `Vagrantfile`
  - P1.3. Desplegar y probar
  - P1.4. Estudio de consumo
- PARTE 2 — Red del laboratorio 3103 con QEMU-KVM (hecha a mano)
  - Visión general
  - Paso 0. Espacio de trabajo y ficheros de Moodle
  - Paso 1. La máquina molde
  - Paso 2. Instalar el software común en el molde
  - Paso 3. Dejar limpia la red del molde y congelarlo
  - Paso 4. Primera red: servidores (SRV ↔ RA)
  - Paso 5. Una VM con varias tarjetas (RA ↔ RB)
  - Paso 6. Rutas estáticas con FRR
  - Paso 7. RB completo y RC
  - Paso 8. PC1 y PC2
  - Paso 9. Rama derecha: RD, PC3 y PC4
  - Paso 10. El anfitrión como PC Profesor
  - Paso 11. Servidor web en SRV
  - Paso 12. Plan de pruebas de conectividad global
- Rutina tras reiniciar el PC
- Siguiente paso: el script de despliegue

Cómo leer esta guía

Esta guía recorre la práctica **en el orden en que se ha hecho**. Para cada paso indica:

- **Objetivo**: qué se quiere conseguir.
- **Órdenes**: exactamente lo que se ejecuta.
- **Explicación**: qué hace cada orden y por qué.
- **Comprobación**: qué debe salir si todo va bien.
- **Problemas encontrados**: errores reales que aparecieron y cómo se resolvieron.

Convenciones:

Prompt	Dónde se ejecuta
`❯` o `$`	En el **anfitrión** (tu PC, Ubuntu 24.04)
`root@<vm>:~#`	**Dentro** de una máquina virtual, entrando con `sudo virsh console <vm>` (usuario `root`, contraseña `agr`)
`RA#`, `RA(config)#`	Dentro de `vtysh`, la consola de FRR en un router

> **Aviso importante (aprendido por las malas):** antes de pegar órdenes de configuración de red o `hostnamectl`, mira el prompt. Si es el de tu PC y no el de la VM, estarías cambiando tu propio ordenador.

Documentos relacionados:

- `requisitos_punto_a_punto.pdf`: cada punto del enunciado y cómo se cumple.
- `red_laboratorio_3103.pdf`: esquema de conexiones, tabla de accesos y plan de pruebas.
- `diagrama_red_lab3103` y `diagrama_construccion_lab3103`: diagramas.

PARTE 1 — Servidor web con Vagrant

P1.1. Instalar VirtualBox y Vagrant

Objetivo: tener el proveedor de virtualización (VirtualBox) y la herramienta que automatiza el despliegue (Vagrant).

```
        # VirtualBox (repositorio de Oracle)
        sudo apt install -y wget gpg build-essential dkms linux-headers-$(uname -r)
        wget -O- https://www.virtualbox.org/download/oracle_vbox_2016.asc \
          | sudo gpg --dearmor --yes -o /usr/share/keyrings/oracle-virtualbox-2016.gpg
        echo "deb [arch=amd64 signed-by=/usr/share/keyrings/oracle-virtualbox-2016.gpg]
https://download.virtualbox.org/virtualbox/debian $(lsb_release -cs) contrib" \
          | sudo tee /etc/apt/sources.list.d/virtualbox.list
        sudo apt update && sudo apt install -y virtualbox-7.2
        sudo usermod -aG vboxusers $USER

        # Vagrant (repositorio de HashiCorp)
        wget -O- https://apt.releases.hashicorp.com/gpg \
          | sudo gpg --dearmor --yes -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
        echo "deb [arch=$(dpkg --print-architecture) signed-
by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com
$(lsb_release -cs) main" \
          | sudo tee /etc/apt/sources.list.d/hashicorp.list
        sudo apt update && sudo apt install -y vagrant
```

Comprobación: `VBoxManage --version` (sin `WARNING`) y `vagrant --version`.

Problemas encontrados en el anfitrión:

Síntoma	Causa	Solución
`vagrant up`: *No usable default provider*	El módulo `vboxdrv` no estaba cargado	Ver filas siguientes
`vboxconfig` no compila	Kernel 7.0 demasiado nuevo para VirtualBox 7.1	Instalar `virtualbox-7.2`
`modprobe vboxdrv failed`	**Secure Boot** activo: el módulo está firmado con una clave no registrada	Desactivar Secure Boot en la BIOS y `sudo /sbin/vboxconfig`
`VERR_VMX_IN_VMX_ROOT_MODE` / `VERR_SVM_IN_USE`	KVM ocupa la virtualización del procesador	Descargar KVM (`sudo modprobe -r kvm_amd kvm`); el `Vagrantfile` lo hace solo

P1.2. El `Vagrantfile`

Objetivo: un único script Ruby que crea la VM, instala Node, descarga el servidor de GitHub, lo arranca y redirige el puerto 8080 → 3000.

Fichero: `parte1-vagrant/Vagrantfile`. Su contenido, por bloques:

Bloque	Qué hace
Parámetros	Box `bento/ubuntu-22.04`, 1 CPU, 1 GB, IPs 192.168.56.10 y 192.168.57.10, puertos 3000/8080, repositorio del servidor
`SCRIPT_SISTEMA`	Teclado español, `es_ES.UTF-8`, zona horaria de Madrid, paquetes básicos, cortafuegos `ufw` (sólo 22 y 3000)
`SCRIPT_NODE`	Node.js 22 desde NodeSource
`SCRIPT_SERVIDOR`	Usuario `nodeapp`, `git clone`, `npm install`, servicio systemd `ssr-server` (`node app.js`)
`SCRIPT_LOGIN_INFO`	Al iniciar sesión se imprime la configuración del servidor
`trigger.before :up`	Descarga KVM y carga VirtualBox antes de arrancar
`forwarded_port`	`host: 8080 → guest: 3000`, en todas las IPs del anfitrión
`private_network` ×2	192.168.56.10 (servicio) y 192.168.57.10 (gestión)
`public_network`	Opcional, con `BRIDGE_IF=<interfaz>`
`provider "virtualbox"`	Nombre, RAM, CPU, clon enlazado, tope de CPU 80 %

Por qué `node app.js` y no `npm start`: `app.js` ya hace `app.listen(3000)` y `bin/www` (lo que lanza `npm start`) vuelve a escuchar en el 3000, así que muere con `EADDRINUSE`. Se comprobó ejecutándolo.

P1.3. Desplegar y probar

```
        cd parte1-vagrant
        vagrant up
        vagrant ssh                     # muestra la configuración del servidor
        curl http://localhost:8080      # desde el anfitrión, por la redirección de
puertos
        curl http://192.168.56.10:3000  # por la red privada
        curl "http://localhost:8080/record?
id_nodo=n1&temperatura=24.5&humedad=60&co2=300&volatiles=100"
```

Comprobación: las páginas devuelven el HTML de *Data-Logger*; `/record` responde `Saving: …` y crea un CSV en `/opt/ssr-master-server/public/logs`.

P1.4. Estudio de consumo

Con la VM encendida se midieron RSS del proceso `VBoxHeadless`, `VBoxManage metrics`, `free`, `df`, `top` y `du`, en reposo y con 25 000 peticiones a `/record`. Datos en bruto en `doc/development/parte1/consumo_raw.txt`; análisis en `memoria_parte1.pdf` (§4): ~1.04 GiB de RAM en el anfitrión por la VM de 1 GB, ~0.06 % de CPU en reposo y ~2.3 % con carga, 2.1 GB de

disco diferencial, estimación para el laboratorio completo y comparación con servidores comerciales.

PARTE 2 — Red del laboratorio 3103 con QEMU-KVM (hecha a mano)

Visión general

El objetivo es reproducir con máquinas virtuales el plano `doc/development/enunciado/plan_numeracion_3103.pdf`:

```
              [SRV] 10.100.0.11
                |  br-srv 10.100.0.0/24
              [RA] --- br-prof 10.200.0.0/30 --- [Anfitrión] 10.200.0.2
                |  br-ab 10.1.0.0/30
              [RB]
   br-bc       /    \       br-bd
 10.2.0.0/30  /      \  10.3.0.0/30
           [RC]      [RD]
           /  \      /  \
   br-red1  br-red2  br-red3  br-red4
      |        |        |        |
    [PC1]    [PC2]    [PC3]    [PC4]
```

Piezas que se usan:

Pieza	Qué es
KVM	Parte del kernel de Linux que usa la virtualización del procesador
QEMU	El programa que "fabrica" cada máquina virtual (disco, tarjetas, consola) y usa KVM
libvirt	El servicio que gestiona las máquinas de QEMU; cada máquina se describe con un XML
virsh	La orden para hablar con libvirt: `define`, `start`, `shutdown`, `destroy`, `console`…
qcow2	Formato de disco que permite discos **diferenciales**: un disco que sólo guarda los cambios respecto a otro
bridge Linux	Un switch virtual en el anfitrión. Cada red del plano es un bridge
tap (`vnetX`)	El "cable" de cada tarjeta de una VM, que libvirt enchufa a su bridge
FRR	La *Software Routing Suite* (sucesora de Quagga) que hace de software de los routers

Preparación previa del anfitrión (sólo una vez):

```
        sudo apt install -y qemu-system-x86 qemu-utils libvirt-daemon-system libvirt-
clients bridge-utils
        sudo systemctl enable --now libvirtd
```

Paso 0. Espacio de trabajo y ficheros de Moodle

Objetivo: dos sitios: uno para lo que escribimos (en el repositorio) y otro para los discos (donde libvirt puede leerlos).

```
        mkdir -p ~/AGR/parte2/xml
        sudo mkdir -p /var/lib/libvirt/images/lab3103
        sudo cp /ruta/a/agr-vm-base.qcow2 /var/lib/libvirt/images/lab3103/base.qcow2
        cp /ruta/a/plantilla_de_moodle.xml ~/AGR/parte2/plantilla.xml
        sudo chmod 444 /var/lib/libvirt/images/lab3103/base.qcow2
        sudo qemu-img info /var/lib/libvirt/images/lab3103/base.qcow2
```

Explicación:

- Los discos van en `/var/lib/libvirt/images/` porque QEMU se ejecuta como el usuario `libvirt-qemu`, que no puede entrar en tu `/home`.
- `chmod 444` pone la imagen de Moodle en sólo lectura: nunca se arranca directamente y así no se puede estropear.

Comprobación: `qemu-img info` → `file format: qcow2`, `virtual size: 4 GiB`. `sudo virsh list --all` vacío.

La plantilla XML de Moodle (base de todas las VMs):

Elemento	Significado
`<domain type="kvm">`	Máquina acelerada con KVM
`<name>XXX</name>`	Nombre de la VM en libvirt (**se cambia**)
`<memory>`/`<currentMemory>` 1048576	1 GB de RAM (en KiB)
`<vcpu>2</vcpu>`	2 CPUs virtuales
`<os>`…`<boot dev="hd"/>`	PC x86-64 que arranca del disco
`<features>` acpi apic pae	ACPI permite el apagado ordenado (`virsh shutdown`)
`<emulator>/usr/bin/kvm`	Programa QEMU que ejecuta la VM
`<disk>` `<source file="/mnt/tmp/XXX/XXX.qcow2"/>`	Disco de la VM (**se cambia**), visto dentro como `/dev/vda`
`<interface type="bridge">` `<source bridge="XXX"/>`	Tarjeta de red enchufada a un bridge (**se cambia**; una por red)
`<serial>` + `<console>`	Consola serie: la que usa `virsh console`
`<graphics type="vnc">`	Pantalla gráfica por VNC

Paso 1. La máquina molde

Objetivo: las 9 VMs necesitan software que la imagen de Moodle no trae (FRR, `traceroute`, Node.js…) y en el laboratorio no tendrán Internet. Se crea **una** VM temporal con Internet, se le instala todo y su disco será la base de las 9.

```
cd /var/lib/libvirt/images/lab3103
sudo qemu-img create -f qcow2 -F qcow2 -b base.qcow2 molde.qcow2
sudo qemu-img info molde.qcow2            # backing file: base.qcow2
cp ~/AGR/parte2/plantilla.xml ~/AGR/parte2/xml/molde.xml
nvim ~/AGR/parte2/xml/molde.xml
```

Tres cambios en `molde.xml`:

```
<name>molde</name>
<source file="/var/lib/libvirt/images/lab3103/molde.qcow2"/>
<interface type="network">
<source network="default"/>
<model type="virtio"/>
</interface>
```

La red `default` de libvirt es un bridge (`virbr0`) con NAT y DHCP: da a la VM una IP 192.168.122.x y salida a Internet a través del anfitrión.

```
sudo virsh net-start default 2>/dev/null; sudo virsh net-list --all
sudo virsh define ~/AGR/parte2/xml/molde.xml
sudo virsh start molde
sudo virsh console molde        # root / agr ; salir con Ctrl+]
```

Comprobación (dentro): `ip -br a` → `eth0` con 192.168.122.x; `ping -c 2 8.8.8.8` responde.

Problema encontrado: `XML error: Missing required attribute 'network' in element 'source'`. El atributo estaba mal escrito (`nwtwork`). Con `type="network"` el `<source>` debe llevar `network="…"`; con `type="bridge"`, `bridge="…"`.

Paso 2. Instalar el software común en el molde

Dentro del molde (`root@agr:~#`).

2A. FRR y herramientas de red

```
apt update
apt install -y frr traceroute tcpdump curl git
systemctl disable --now frr
systemctl is-enabled frr        # disabled
```

Paquete	Uso
`frr`	*Software Routing Suite*: `zebra` (rutas del kernel), `staticd` (rutas estáticas), `vtysh` (consola tipo Cisco). Versión 7.2.1
`traceroute`, `tcpdump`, `curl`	Pruebas: camino de los paquetes, captura de tráfico, peticiones HTTP
`git`	Descargar el servidor web

FRR se deja **desactivado**: sólo se activará en los 4 routers.

2B. Node.js y el servidor web (reciclado de la parte 1)

```
        curl -fsSL https://deb.nodesource.com/setup_22.x | bash -
        apt install -y nodejs
        useradd --system --create-home --shell /usr/sbin/nologin nodeapp
        git clone --depth 1 https://github.com/gisai/SSR-master-server.git /opt/ssr-
master-server
        cd /opt/ssr-master-server
        mkdir -p public/logs
        npm install --omit=dev
        chown -R nodeapp:nodeapp /opt/ssr-master-server

        cat > /etc/systemd/system/ssr-server.service <<'EOF'
        [Unit]
        Description=SSR-master-server (Node.js Data-Logger)
        After=network.target

        [Service]
        User=nodeapp
        WorkingDirectory=/opt/ssr-master-server
        ExecStart=/usr/bin/node app.js
        Restart=on-failure

        [Install]
        WantedBy=multi-user.target
        EOF
        systemctl daemon-reload
        systemctl start ssr-server; sleep 3; curl -s localhost:3000 | head -5
        systemctl stop ssr-server
        systemctl is-enabled ssr-server   # disabled
```

Comprobación: `node -v` → `v22.x`; `curl` devuelve el HTML de *Data-Logger*. El servicio queda **desactivado**: sólo se activará en SRV.

Paso 3. Dejar limpia la red del molde y congelarlo

Objetivo: la imagen de Moodle pide IP por DHCP en `eth0`. En el laboratorio no hay DHCP: cada VM tendrá IPs fijas.

Inspección (dentro del molde):

```
ls /etc/netplan/; cat /etc/netplan/*.yaml
cat /etc/network/interfaces
dpkg -l ifupdown | tail -1
systemctl is-active systemd-networkd networking
ls /etc/cloud/
```

Hallazgo	Conclusión
netplan configura `ens3` con MAC `52:54:00:12:34:56`	No se usa (esa MAC no es la nuestra)
`ifupdown` instalado, `/etc/network/interfaces` con `eth0` por DHCP	**Es el que manda**
`/etc/cloud/` sólo con `build.info`	cloud-init no está: nada reescribe la red
`/proc/cmdline` con `net.ifnames=0`	Las tarjetas se llaman `eth0`, `eth1`, `eth2` **en el orden del XML**

Limpieza y apagado:

```
rm /etc/netplan/50-cloud-init.yaml
cat > /etc/network/interfaces <<'EOF'
auto lo
iface lo inet loopback
EOF
apt clean
truncate -s 0 /etc/machine-id     # cada VM generará el suyo
poweroff
```

Congelar (en el anfitrión):

```
sudo virsh undefine molde
sudo chmod 444 /var/lib/libvirt/images/lab3103/molde.qcow2
```

Por qué congelarlo: las 9 VMs tendrán discos apoyados en `molde.qcow2`; si el molde cambiara, sus discos quedarían corruptos. Cadena resultante:

```
base.qcow2 (Moodle) ← molde.qcow2 (+ FRR, Node, servidor) ← srv, ra, rb, rc, rd, pc1…pc4
.qcow2
```

Paso 4. Primera red: servidores (SRV ↔ RA)

Objetivo: la red 10.100.0.0/24 con SRV (10.100.0.11) y RA (10.100.0.1).

4.1. El switch

```
sudo ip link add br-srv type bridge
sudo ip link set br-srv up
sudo sysctl -w net.bridge.bridge-nf-call-iptables=0
```

El bridge no lleva IP: el anfitrión sólo presta el switch. El `sysctl` impide que el cortafuegos del anfitrión (Docker pone `FORWARD DROP`) bloquee el tráfico entre VMs.

4.2. Discos y XML

```
cd /var/lib/libvirt/images/lab3103
sudo qemu-img create -f qcow2 -F qcow2 -b molde.qcow2 srv.qcow2
sudo qemu-img create -f qcow2 -F qcow2 -b molde.qcow2 ra.qcow2
cd ~/AGR/parte2
for vm in srv ra; do
  sed -e "s|<name>XXX</name>|<name>$vm</name>|" \
      -e "s|/mnt/tmp/XXX/XXX.qcow2|/var/lib/libvirt/images/lab3103/$vm.qcow2|" \
      -e 's|<source bridge="XXX"/>|<source bridge="br-srv"/>|' \
      plantilla.xml > xml/$vm.xml
done
sudo virsh define xml/srv.xml; sudo virsh define xml/ra.xml
sudo virsh start srv;          sudo virsh start ra
bridge link                    # dos vnetX con master br-srv
```

Los tres `sed` son exactamente los tres cambios a la plantilla: nombre, disco y bridge.

4.3. Configuración dentro de las VMs

SRV:

```
hostnamectl set-hostname srv
cat > /etc/network/interfaces <<'EOF'
auto lo
iface lo inet loopback

# Red de servidores 10.100.0.0/24
auto eth0
iface eth0 inet static
    address 10.100.0.11/24
EOF
systemctl restart networking
```

RA (igual, con `ra` y `address 10.100.0.1/24`, comentario `# ge0/0/3`).

Comprobación: desde RA `ping -c 3 10.100.0.11` → 0 % de pérdida, y al revés.

Problemas encontrados:

- Las órdenes de configuración se ejecutaron por error en el anfitrión (prompt `›`): `hostnamectl` renombró el PC (se restauró con el nombre de `/etc/hosts`) y la escritura en `/etc/network/interfaces` falló por permisos (`sudo cat > fichero` no sirve: la redirección la hace la shell sin privilegios).
- Al volver a la consola, la sesión seguía abierta y `root/agr` se ejecutaron como órdenes ("Command not found"). Pulsar `Enter` y mirar el prompt antes de escribir; salir con `exit` + `Ctrl+]`.

Paso 5. Una VM con varias tarjetas (RA ↔ RB)

Objetivo: el enlace RA ge0/0/1 (10.1.0.1/30) — RB ge0/0/5 (10.1.0.2/30).

```
sudo ip link add br-ab type bridge; sudo ip link set br-ab up
```

Segunda tarjeta en RA: en `xml/ra.xml`, debajo del bloque de `br-srv`:

```
<interface type="bridge">
<source bridge="br-ab"/>
<model type="virtio"/>
</interface>
```

Aplicar un cambio de XML a una VM que ya existe (procedimiento fijo):

```
sudo virsh shutdown ra
until sudo virsh domstate ra | grep -q 'shut off'; do sleep 1; done
sudo virsh undefine ra
sudo virsh define ~/AGR/parte2/xml/ra.xml
sudo virsh start ra
sudo virsh domiflist ra          # br-srv y br-ab
```

RB se crea como SRV/RA con `br-ab`. Dentro de RA se **añade** `eth1` (con `cat >> /etc/network/interfaces`, dos `>>` para no borrar `eth0`); dentro de RB se configura `eth0 10.1.0.2/30`.

Comprobación: RB → 10.1.0.1 responde; RB → 10.100.0.11 da *Network is unreachable* (esperado: aún no hay rutas).

Problemas encontrados:

- El bloque de `br-srv` quedó duplicado al editar → se borró uno.
- `virsh define` sobre una VM existente: *domain 'ra' already exists with uuid* porque nuestro XML no lleva UUID → `undefine` antes de `define` (el disco no se toca).
- `virsh start` lanzado antes de terminar el `shutdown` arrancó la definición vieja → esperar a `shut off`.

Paso 6. Rutas estáticas con FRR

Objetivo: que RB llegue a SRV. Un equipo sólo conoce las redes donde tiene tarjeta; para el resto necesita **rutas** y los routers deben **reenviar**.

Equipo	Ruta	Del plano
RA	0.0.0.0/0 → 10.1.0.2	Tabla de RA
RB	0.0.0.0/0 → 10.1.0.1	Tabla de RB
SRV	puerta de enlace 10.100.0.1	—

En cada router (RA y RB):

```
echo 'net.ipv4.ip_forward=1' > /etc/sysctl.d/99-router.conf
sysctl -p /etc/sysctl.d/99-router.conf
systemctl enable --now frr
vtysh
```

```
configure terminal
hostname RA
ip route 0.0.0.0/0 10.1.0.2
end
write memory
show ip route
exit
```

(En RB: `hostname RB` e `ip route 0.0.0.0/0 10.1.0.1`.) `write memory` guarda en `/etc/frr/frr.conf`. `staticd` arranca siempre en FRR 7.2 aunque no aparezca en `/etc/frr/daemons`.

En SRV se añade `gateway 10.100.0.1` a su `eth0` y `systemctl restart networking`.

Comprobación: desde RB, `ping 10.100.0.11` responde con `ttl=63` (un router) y `traceroute -n 10.100.0.11` → `10.1.0.1` → `10.100.0.11`.

Paso 7. RB completo y RC

Objetivo: las tres tarjetas de RB y el router RC con RED 1 y RED 2.

Se crean los bridges con un primer script, `~/AGR/parte2/red.sh`, que se ejecuta tras cada reinicio del PC (los bridges no son permanentes):

```
#!/bin/bash
# red.sh - Prepara el anfitrión para el laboratorio 3103 (tras cada reinicio)
set -e

# 1. KVM sí, VirtualBox no
sudo modprobe -r vboxnetadp vboxnetflt vboxdrv 2>/dev/null || true
sudo modprobe kvm_amd

# 2. Un bridge por cada red del plano
BRIDGES="br-srv br-prof br-ab br-bc br-bd br-red1 br-red2 br-red3 br-red4"
for br in $BRIDGES; do
    if ! ip link show "$br" >/dev/null 2>&1; then
        sudo ip link add "$br" type bridge
    fi
    sudo ip link set "$br" up
done

# 3. Que el cortafuegos (Docker) no filtre el tráfico entre máquinas
sudo sysctl -qw net.bridge.bridge-nf-call-iptables=0 2>/dev/null || true

# 4. El anfitrión es el PC Profesor (10.200.0.2) con rutas al laboratorio vía RA
sudo ip addr replace 10.200.0.2/30 dev br-prof
for net in 10.100.0.0/24 10.1.0.0/30 10.2.0.0/30 10.3.0.0/30 \
           10.0.1.0/24 10.0.2.0/24 10.0.3.0/24 10.0.4.0/24; do
    sudo ip route replace $net via 10.200.0.1
done

echo "Bridges listos:"
ip -br link show type bridge | grep '^br-'
```

(El bloque 4 se añadió en el paso 10.)

RB: XML con tres tarjetas en orden `br-ab`, `br-bc`, `br-bd` → `eth0`, `eth1`, `eth2`; procedimiento shutdown/undefine/define/start. Dentro:

```
        cat >> /etc/network/interfaces <<'EOF'

        # ge0/0/1 - Enlace RB-RC 10.2.0.0/30
        auto eth1
        iface eth1 inet static
            address 10.2.0.1/30

        # ge0/0/4 - Enlace RB-RD 10.3.0.0/30
        auto eth2
        iface eth2 inet static
            address 10.3.0.1/30
        EOF
        systemctl restart networking
        systemctl restart frr
```

```
configure terminal
ip route 10.0.1.0/24 10.2.0.2
ip route 10.0.2.0/24 10.2.0.2
ip route 10.0.3.0/24 10.3.0.2
ip route 10.0.4.0/24 10.3.0.2
end
write memory
```

RC: disco sobre el molde; XML con `br-bc`, `br-red1`, `br-red2`; dentro: `hostname rc`, `eth0 10.2.0.2/30`, `eth1 10.0.1.1/24`, `eth2 10.0.2.1/24`, reenvío IP, FRR con `hostname RC` e `ip route 0.0.0.0/0 10.2.0.1`.

Comprobación: desde RC, `traceroute -n 10.100.0.11` → `10.2.0.1` (RB) → `10.1.0.1` (RA) → `10.100.0.11`; `ping` con `ttl=62` (dos routers).

Problema encontrado: el primer ping de RC a SRV dio *Destination Net Unreachable* desde RB. Al hacer `systemctl restart networking` en RB, el kernel borró la ruta por defecto instalada por FRR (sale por `eth0`, cuya IP se quitó y se volvió a poner) y FRR 7.2 no la reinstaló aunque `show ip route` la seguía mostrando. Solución: `systemctl restart frr`. **Regla:** en un router, después de reiniciar la red, reiniciar FRR.

Paso 8. PC1 y PC2

```
        cd /var/lib/libvirt/images/lab3103
        for n in 1 2; do sudo qemu-img create -f qcow2 -F qcow2 -b molde.qcow2 pc$n.qcow2;
done
        cd ~/AGR/parte2
        for n in 1 2; do
          sed -e "s|<name>XXX</name>|<name>pc$n</name>|" \
              -e "s|/mnt/tmp/XXX/XXX.qcow2|/var/lib/libvirt/images/lab3103/pc$n.qcow2|" \
              -e "s|<source bridge=\"XXX\"/>|<source bridge=\"br-red$n\"/>|" \
              plantilla.xml > xml/pc$n.xml
          sudo virsh define xml/pc$n.xml; sudo virsh start pc$n
        done
```

Dentro de cada PC (ejemplo PC1):

```
        hostnamectl set-hostname pc1
        cat > /etc/network/interfaces <<'EOF'
        auto lo
        iface lo inet loopback

        # RED 1 10.0.1.0/24
        auto eth0
        iface eth0 inet static
            address 10.0.1.10/24
            gateway 10.0.1.1
        EOF
        systemctl restart networking
```

Un PC no lleva FRR ni reenvío: sólo IP y puerta de enlace.

Comprobación (PC2): → 10.0.2.1 (`ttl=64`), → PC1 (`ttl=63`), → SRV (`ttl=61`); `traceroute` → `10.0.2.1` → `10.2.0.1` → `10.1.0.1` → `10.100.0.11`.

Aclaración: `virsh list --all` **sin** `sudo` sale vacío porque se conecta a `qemu:///session` (un libvirt privado del usuario), no a `qemu:///system`.

Paso 9. Rama derecha: RD, PC3 y PC4

RD se creó sin abrir el editor, añadiendo las dos tarjetas con el propio `sed`:

```
        cd /var/lib/libvirt/images/lab3103
        sudo qemu-img create -f qcow2 -F qcow2 -b molde.qcow2 rd.qcow2
        cd ~/AGR/parte2
        sed -e 's|<name>XXX</name>|<name>rd</name>|' \
            -e 's|/mnt/tmp/XXX/XXX.qcow2|/var/lib/libvirt/images/lab3103/rd.qcow2|' \
            -e 's|<source bridge="XXX"/>|<source bridge="br-bd"/>|' \
            -e 's|</interface>|</interface>\n<interface type="bridge">\n<source
bridge="br-red3"/>\n<model type="virtio"/>\n</interface>\n<interface
type="bridge">\n<source bridge="br-red4"/>\n<model type="virtio"/>\n</interface>|' \
            plantilla.xml > xml/rd.xml
        sudo virsh define xml/rd.xml; sudo virsh start rd
```

Dentro de RD: `hostname rd`; `eth0 10.3.0.2/30`, `eth1 10.0.3.1/24`, `eth2 10.0.4.1/24`; reenvío; FRR con `hostname RD` e `ip route 0.0.0.0/0 10.3.0.1`. PC3 y PC4 como PC1 (`10.0.3.10` gw `10.0.3.1`, `10.0.4.10` gw `10.0.4.1`).

Problema encontrado: desde PC3, `ping 10.0.3.1` → *Destination Host Unreachable* emitido por la propia PC3: nadie respondía al ARP de la puerta de enlace porque **RD no existía**. Se creó RD y se resolvió. También `virsh console pc4` → *failed to get domain*: PC4 no estaba definida.

Comprobación (PC4): → RD, PC3 (`ttl=63`), PC1 (`ttl=61`), SRV (`ttl=61`); `traceroute` → `10.0.4.1` → `10.3.0.1` → `10.1.0.1` → `10.100.0.11`.

Paso 10. El anfitrión como PC Profesor

Objetivo: «La máquina anfitriona también deberá tener acceso al servidor». Se sigue el plano: el PC Profesor cuelga de RA ge0/0/5 en 10.200.0.0/30.

1. Tercera tarjeta en RA (`br-prof`) con el procedimiento de cambio de XML.
2. Dentro de RA: `eth2 10.200.0.1/30` añadida a `/etc/network/interfaces`, `systemctl restart networking` **y** `systemctl restart frr`.
3. En el anfitrión:

```
        sudo ip addr add 10.200.0.2/30 dev br-prof
        for net in 10.100.0.0/24 10.1.0.0/30 10.2.0.0/30 10.3.0.0/30 \
                   10.0.1.0/24 10.0.2.0/24 10.0.3.0/24 10.0.4.0/24; do
          sudo ip route add $net via 10.200.0.1
        done
```

Sólo las redes del laboratorio van por RA; Internet sigue saliendo por la tarjeta real del PC (`enp8s0`, 10.200.78.15/24, que no se solapa con ninguna red del laboratorio). La vuelta no necesita nada: todos los routers mandan lo desconocido hacia RA y RA tiene 10.200.0.0/30 conectada. Este bloque se añadió a `red.sh` con `replace` para que sea repetible.

Paso 11. Servidor web en SRV

```
        # en SRV
        systemctl enable --now ssr-server
        ss -tlnp | grep 3000          # LISTEN *:3000 node
```

Pruebas:

```
        # en PC1 / PC4
        curl -s http://10.100.0.11:3000/ | head -4
        curl -s "http://10.100.0.11:3000/record?
id_nodo=pc1&temperatura=21.5&humedad=40&co2=400&volatiles=10"
        # en el anfitrión
        curl -s http://10.100.0.11:3000/ | head -4
        curl -s "http://10.100.0.11:3000/record?
id_nodo=profesor&temperatura=20&humedad=50&co2=420&volatiles=8"
```

Comprobación: en SRV, `/opt/ssr-master-server/public/logs/` contiene `pc1-…csv`, `pc4-…csv` y `profesor-…csv` con los datos enviados.

Observaciones: un `ss` lanzado 3 ms después de arrancar no mostró el puerto (Node aún no escuchaba); el mes del nombre del CSV sale 9 en octubre porque el servidor usa `getMonth()` (meses desde 0).

Paso 12. Plan de pruebas de conectividad global

En los 10 equipos (9 VMs y el anfitrión):

```
        H=$(hostname); ok=0; fail=""; for ip in 10.100.0.11 10.100.0.1 10.200.0.1
10.200.0.2 10.1.0.1 10.1.0.2 10.2.0.1 10.2.0.2 10.3.0.1 10.3.0.2 10.0.1.1 10.0.1.10
10.0.2.1 10.0.2.10 10.0.3.1 10.0.3.10 10.0.4.1 10.0.4.10; do if ping -c1 -W1 $ip
>/dev/null 2>&1; then ok=$((ok+1)); else fail="$fail $ip"; fi; done; web=$(curl -s -o
/dev/null -w '%{http_code}' --max-time 5 http://10.100.0.11:3000/); echo "$H: $ok/18 OK,
web=$web, fallan:${fail:- ninguna}"
```

Resultado: los 10 equipos → `18/18 OK, web=200, fallan: ninguna` (180/180 pings, 10/10 accesos web). Tabla completa de accesos y caminos en `red_laboratorio_3103.pdf`.

Rutina tras reiniciar el PC

```
        ~/AGR/parte2/red.sh
        for vm in srv ra rb rc rd pc1 pc2 pc3 pc4; do sudo virsh start $vm; done
        sudo virsh list --all
```

Lo que hay **dentro** de las VMs (IPs, FRR, servidor web) está en sus discos y se conserva; lo que hay en el **anfitrión** (bridges, IP de `br-prof`, rutas, `sysctl`) se pierde y lo rehace `red.sh`.

Siguiente paso: el script de despliegue

El enunciado pide «desarrollar un script que despliegue una red virtualizada» y que «deberá generar los ficheros XML necesarios». Todo lo de los pasos 0–11 es lo que el script tiene que automatizar:

Paso manual	Lo que hará el script
0–3	Preparar `base.qcow2` y la imagen molde (o reutilizarla si ya existe)
4–10 (bridges)	Crear los 9 bridges, la IP del anfitrión y sus rutas (`red.sh`)
4–9 (discos)	`qemu-img create -b molde.qcow2` por VM
4–9 (XML)	Generar cada XML desde la plantilla de Moodle (con `lxml`, como aconseja el enunciado)
4–9 (`virsh`)	`define` + `start`; `shutdown`/`destroy` para pararlas
4–11 (dentro)	Escribir en cada disco `hostname`, `/etc/network/interfaces`, `99-router.conf`, `frr.conf` y activar `frr`/`ssr-server` antes de arrancar
12	Ejecutar el plan de pruebas y guardar el informe

Todos los ficheros que usa ese script ya están en `parte2/`: la plantilla, los XML de cada VM, `red.sh`, la preparación del molde (`molde/`) y la configuración interna de cada VM (`vm/<vm>/`: `hostname`, `interfaces`, `99-router.conf`, `frr.conf`).

(Una primera versión automatizada con otro diseño se conserva como material de trabajo en `doc/development/historico_script/`.)